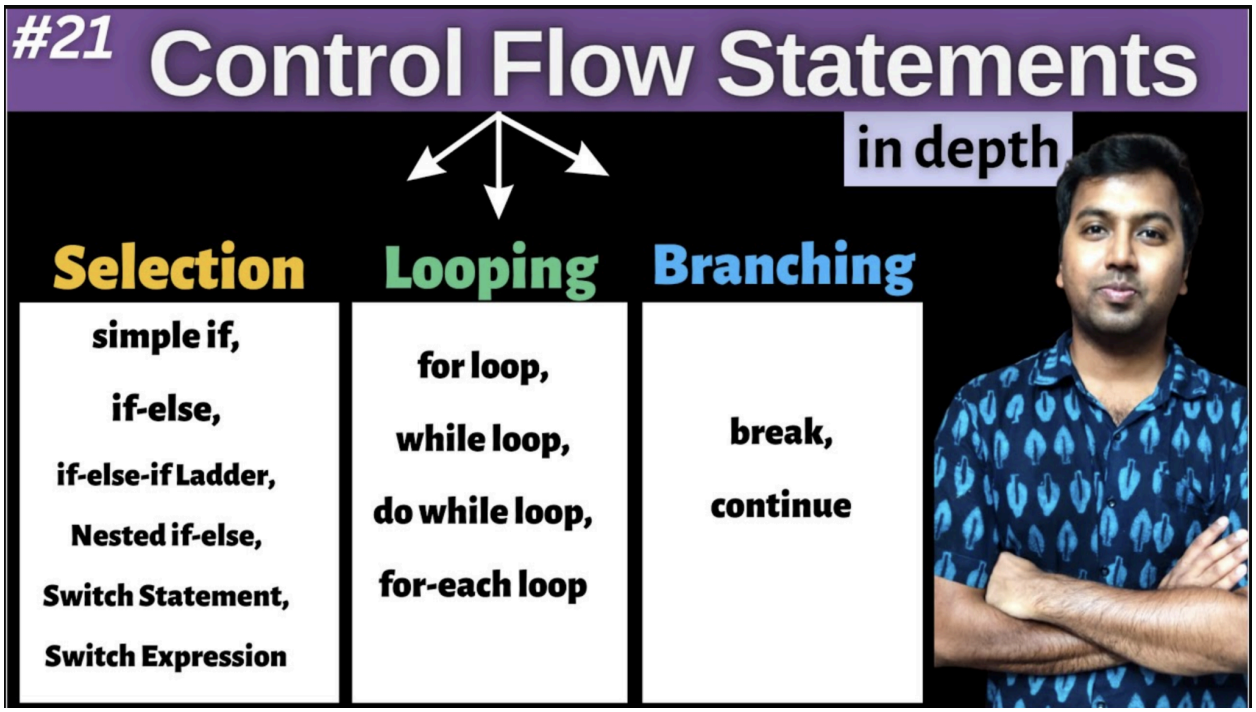
We have already covered Switch statement



Supported datatypes:

- 4 primitive types : int, short, byte, char
- Wrapper types of above primitives: Integer, Short, Byte, Character
- Enum
- String

Java 21: Pattern Matching for Switch

It supports all, what classic Switch can. No change in that

Supported datatypes:

- **4 primitive types : int, short, byte, char**
- **Wrapper types of above primitives: Integer, Short, Byte, Character**
- **Enum**
- **String**

Pattern matching for Objects i.e. classes, interfaces, abstract type, Wrapper Object

Example:

```
if (obj instanceof String s) {
  System.out.println("String: " + s);
}
else if (obj instanceof Integer i) {
  System.out.println("Integer: " + i);
}
else {
  System.out.println("Other type");
}
```

Internally it might be doing similar to what Pattern matching of 'Instanceof' does

Scope of pattern variable is within the block only:

It **cannot be accessed** in:

- Other case blocks
- Default block or
- After the switch

Classes and inheritance:

That's, one of the reason why Pattern matching on Switch is slower than compared to Classic switch (uses Jump table / Lookup Table instead of if-else)

Null handling:

Pattern matching switch is **null-safe**: default handles null automatically.

Grouping Pattern:

Its not possible to group multiple pattern:

Guarded Pattern :

Helps to add addition conditional on Pattern matching using '**when**'

Pattern Matching with Enum

Without Pattern Matching:

With Pattern Matching:

or